

The Alcesat sub-granule index

what is stored, where, and in which columns

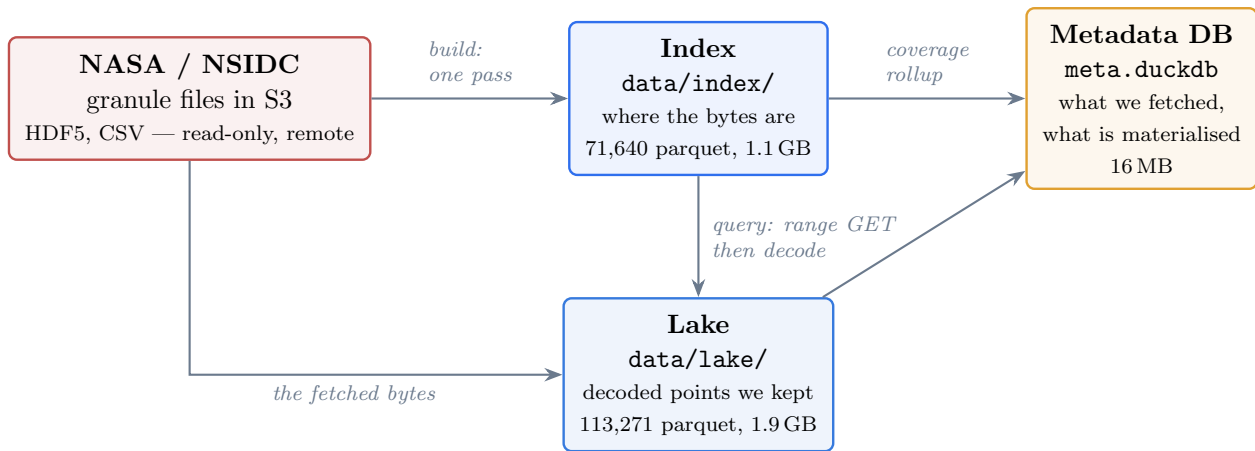
2026-09-09

Abstract

Alcesat answers “give me every altimetry point from four missions inside this box” without downloading whole granules. It does that with a **sub-granule index**: a table recording, for each spatial cell of each granule, the exact *byte ranges* inside the remote file that hold that cell’s points. A query becomes a handful of HTTP range GETs instead of a set of file downloads. This document covers the three stores that make that work, the H3 grid they are keyed on, the chunk map that turns an HDF5 file into addressable pieces, and the contents of the two kinds of Parquet file. Section 7 and Section 8 are open design questions. All measurements are from the deployed `us-west-2` instance, 2026-09-09.

1 The three stores

Three things live on disk, with sharply different jobs. Confusing them has been the source of most of this subsystem’s bugs.



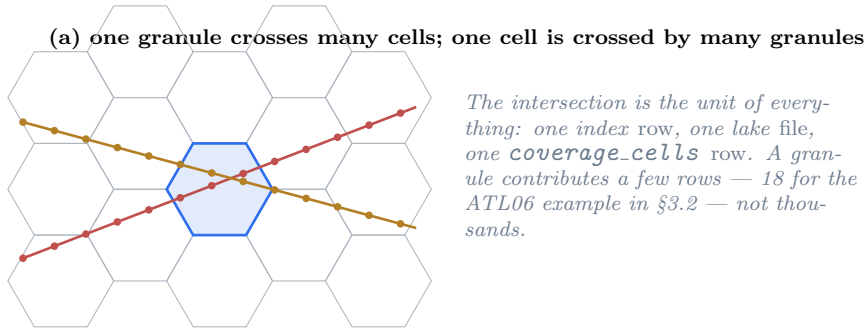
The index is an *address book*: never science values, only the coordinates of bytes. Built once per granule, then immutable. Unit: (granule, beam, chunk, cell).

The lake is a *cache of decoded points*. Losing it costs time, never truth — anything missing is re-fetched. Unit: one Parquet file per (granule, beam, chunk) *inside* one cell directory.

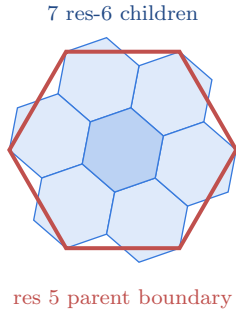
The metadata DB records *what has been fetched* (so a chunk is never fetched twice) and *what is materialised* (so sizing the lake need not walk it). Two different questions — §5.

2 The H3 grid

Every store is keyed on an H3 cell index: a 64-bit integer naming one hexagon on a global grid.



(b) resolution 5 $\rightarrow$ 6: the IDs nest exactly, the geometry does not



H3 subdivides by aperture 7 with a $\approx 19.1^\circ$ twist, so six of the seven children straddle the parent's boundary: the areas match, the shapes do not. What is exact is the index relation — `cellToParent(c,r)` is a total function on cell IDs, so every cell has exactly one ancestor per coarser resolution. A rollup defined over cell IDs is therefore well defined; only one read as “the metadata of the parent's area” inherits the boundary error. §7 rests on this distinction.

Collection	Product	Res	Cells on the globe	Mean area	Mean edge
ATL06	ICESat-2 ATL06 land ice	5	2,016,842	252.9 km ²	9.85 km
GLAS	ICESat/GLAS GLAH06	5	2,016,842	252.9 km ²	9.85 km
ICESSN	IceBridge ATM ILATM2	5	2,016,842	252.9 km ²	9.85 km
ATL03	ICESat-2 ATL03 photons	6	14,117,882	36.1 km ²	3.73 km

The dense photon product is indexed one resolution finer because a res-5 cell holds far too many photons to be a useful fetch unit. Within a collection the index and lake resolutions are deliberately *equal*: a lake file is written at the same resolution its index row named, so a later query for a different sub-box inside that cell re-filters correctly instead of returning a partial cell.

3 The index

3.1 Directory layout

```

data/index/
├── atl06/res5/
│   ├── ATL06_20181014050737_02380104_007_01.h5.parquet one per granule (33,064)
│   ├── ...
│   ├── _coverage/manifest.parquet rolled-up (cell, granule, ym)
│   ├── _coverage/meta.json freshness stamp
│   └── _build.json the cell list this build targets
├── glas/res5/ 4,741 granule parquet + _coverage/
├── icesn/res5/ 33,793 granule parquet + _coverage/
├── atl03/ 42 granule parquet + _coverage/ (no res subdir)
└── meta.duckdb 16 MB

```

Each collection directory is *flat*: one Parquet file per granule, named `<granule>.parquet` with the granule's own `.h5` / `.csv` suffix retained. Two consequences, both of which have bitten:

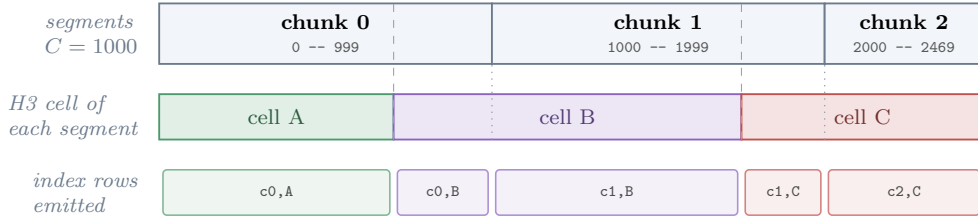
1. Files land **tmp-then-rename**, so a reader never sees a torn Parquet and the *directory* mtime moves whenever a granule arrives. That single `stat()` is the freshness gate used everywhere downstream (§3.5).

2. Anything that answers a question by opening every file does work proportional to the *store*, not the *request* — 43.5s at 33,064 files. The `_coverage/` rollup exists to stop that, and sits in a *subdirectory* so a `*.parquet` glob cannot pick up a schema-mismatched file and so writing it does not invalidate its own freshness stamp.

3.2 The chunk map: from an HDF5 file to index rows

An HDF5 dataset is stored as fixed-length chunks along its first axis, each compressed independently and placed wherever the writer had room. The *chunk map* is the dataset’s B-tree read out as a list: for chunk k , its byte offset, its compressed size, and its filter mask. `_chunk_manifest` walks that B-tree once per dataset — the only time a granule’s B-trees are ever read — and sorts the result into *logical* order, so entry k is the k -th chunk along the axis regardless of where it sits in the file.

(a) one beam of one granule: chunk boundaries and cell boundaries do not line up



The builder emits one row per distinct (chunk, cell) pair. A chunk straddling a cell boundary yields two rows that address the same bytes with different `seg_start/seg_end` and different bounding boxes; a cell spanning two chunks yields two rows addressing different bytes. Both happen constantly, and both are correct: the fetch path de-duplicates ranges before issuing them.

(b) the chunk map: five datasets, one shared chunking, one column per decode unit

dataset	chunk 0	chunk 1	chunk 2
latitude	8555110 / 2715	... / ...	... / ...
longitude	8557825 / 2866	... / ...	... / ...
h_li	8546020 / 482	... / ...	... / ...
delta.time	8544140 / 1880	... / ...	... / ...
atl06.quality.summary	8544049 / 91	... / ...	... / ...

Each cell above holds `byte_offset` and `size`, plus `filters`, `dtype`, `ncols` and `filter_mask`. One column of this table = one index row’s address group = five byte ranges. All five datasets of a beam are asserted at build to share the chunk length C and the chunk count, so `chunk_index` names the same segment span in every one of them — that assertion is what makes a single integer a sufficient decode unit. Without it the row would need a separate chunk number per dataset.

Chunk numbering is logical, byte placement is not: the five ranges of one chunk usually sit close together (they were written together) but the chunks of one dataset are not necessarily in file order, and ATL03’s ranges can be gigabytes apart. That is why the fetch path sorts and coalesces ranges with a gap threshold rather than assuming adjacency — §3.4.

3.3 What is in one index Parquet file

`data/index/atl06/res5/ATL06.20181014050737.02380104.007.01.h5.parquet` — 18 rows, one per (beam, chunk, cell)

IDENTITY	WHERE	MISSION	ADDRESS × 5 datasets
granule url s3url revision	h3.cell lat_min lat_max lon_min lon_max chunk_index seg_start seg_end	sc.orient sdp_epoch beam strong cycle rgt	latitude_offset latitude_size latitude_filters latitude_dtype latitude_ncols latitude_mask the same sextet again for longitude, h_li, delta_time, atl06_quality_summary
file metadata: aicesat_atl06_index.version, h3.res=5, built.at, cells			

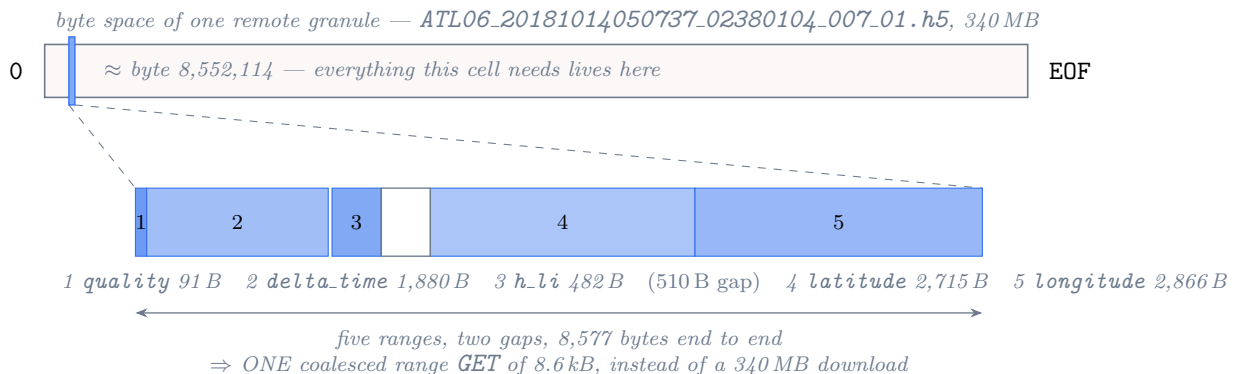
Four column groups. IDENTITY is the join key plus the two ways to reach the bytes (`s3url` in `us-west-2`: no presign, free egress; `url` elsewhere, via presigned redirect). WHERE is everything needed to decide whether a row is wanted, without touching the network. MISSION carries the per-granule constants a decoder needs. ADDRESS is one column of the chunk map — §3.2. 33,064 such files, 1.1 GB, for ATL06.

Only the ADDRESS group differs between collections:

Collection	Row grain	Address columns
ATL06, ATL03	(granule, beam, chunk, cell)	per dataset: <code>_offset _size _filters _dtype _ncols _mask</code> . HDF5 chunk ranges. <code>_ncols</code> matters: ATL03's <code>signal.conf_ph</code> is $n \times 5$.
GLAS	(granule, beam=na, chunk, cell)	the same, with <code>_fill</code> instead of <code>_ncols</code> : GLAH06 encodes no-data as 1.798×10^{308} , not NaN.
ICESSN	(granule, cell)	<code>byte_start byte_end n_lines</code> . Plain CSV — a line range, no chunk map at all.

GLAS and ICESSN replace the MISSION group with `gdate` (YYYYMMDD), having no repeat-cycle number; ATL03 replaces `seg_start/seg_end` with `ph_start/ph_end`. Row counts per granule stay small — 18 (ATL06), 7 (ATL03), 25 (GLAS), 2 (ICESSN) — so the whole index is 1.1 GB across 71,640 granules.

3.4 Byte-range addressing



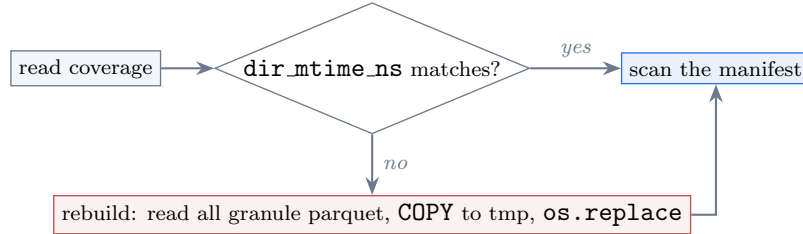
the index row that says so

```
h3_cell=598985434483130367 beam=gt1l chunk_index=0
seg_start=0 seg_end=410 lat_min=80.860 lat_max=81.257
latitude_offset=8555110 latitude_size=2715
latitude_filters=gzip latitude_dtype=float64 latitude_ncols=1
```

`offset/size` give the range; `filters` says how to inflate it (`gzip`, or `shuffle`, `gzip` for the `int8` columns); `dtype/ncols` say how to view the result. Nothing here requires opening the file. ATL03's ranges are not adjacent — 3.28 GB apart in the example granule — which is why coalescing is bounded by a gap threshold rather than unconditional.

3.5 The coverage manifest

Answering “which cells does this collection cover, over what period?” from the per-granule files means opening all of them. `<index dir>/_coverage/manifest.parquet` is the rollup that avoids it: the DISTINCT triples `h3.cell` (uint64), `granule` (string), `ym` (YYYY-MM, from `gdate` for GLAS/ICESSN, parsed out of the granule *name* for ATL0x). For ATL06 that is 2,261,439 rows in 27 MB, against 33,064 files and 1.1 GB.



The companion `meta.json` holds `{n_source_files, dir_mtime_ns, built_at}`. The stamp is written before the sources are read, so a granule landing mid-build leaves the recorded `mtime` older than the directory’s and the next read rebuilds — stale-and-corrected, never silently-missing. Writes land inside `_coverage/`, which does not touch the parent’s `mtime`, so the manifest cannot invalidate itself.

Known lag. The rebuild being lazy, the manifest trails the directory by a few granules between builds: 33,053 vs 33,064 (ATL06), 4,649 vs 4,741 (GLAS), 33,578 vs 33,793 (ICESSN). Those granules fall in already-covered cells, so the *cell set* is unaffected (14,482 / 23,142 / 7,782, identical either way). Counts that must be exact — notably the “index build *N*% done” readout — come from a directory listing instead.

4 The lake

4.1 Layout and naming

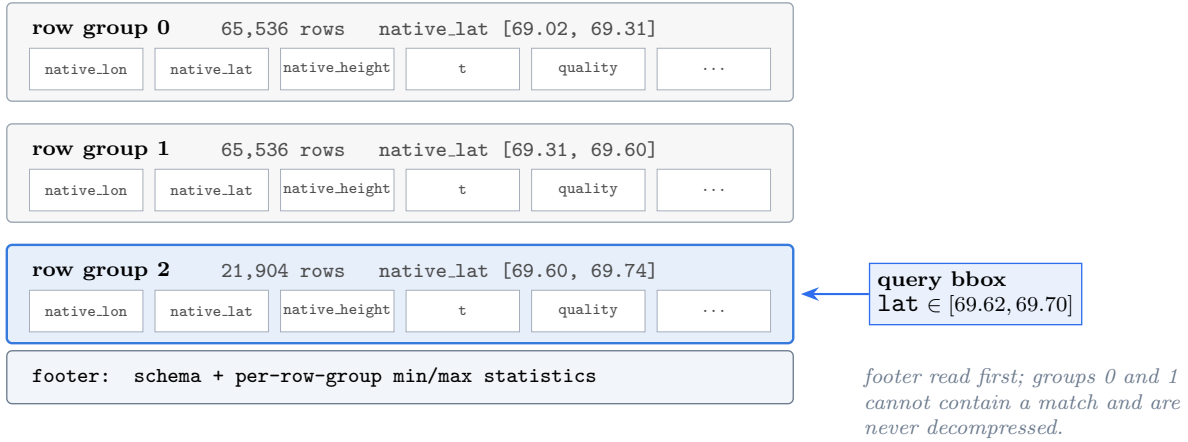
```
data/lake/mission=ATL06/h3_cell=599100951923523583/
  ATL06_20181119053215_07880105_007_01__gt1l__c5.parquet  <- granule __ beam __ chunk
  ATL06_20181119053215_07880105_007_01__gt1r__c5.parquet
  ... (656 files in this cell)
data/lake/mission=ICESAT2/h3_cell=604538106045005823/
  ATL03_20210830193051_10411206_007_01.h5__gt2r.parquet  <- granule __ beam (NO chunk)
```

Hive-style partitioning, so DuckDB prunes on `mission=` and `h3_cell=` without reading anything. The file name is the write’s identity, which makes re-materialisation idempotent by construction.

The photon path names files differently, and it matters. `write_point.chunk` (ATL06 / GLAS / ICESSN) emits one file per (granule, beam, chunk), so one coverage row is exactly one file. `write_photons` (ATL03, under `mission=ICESAT2`) omits the chunk, so *many* coverage rows map to *one* file: 390 rows against 363 files. Any scheme deriving per-file quantities by summing per-row quantities is wrong on this path.

4.2 What is in one lake Parquet file

mission=ATL06/h3_cell=599100951923523583/ATL06_..._007_01__gt1l__c5.parquet



Rows are written in along-track order at `ROW_GROUP_ROWS = 65,536`, so each group carries tight lat/lon statistics and the pruning above works; without that ordering every group would span the whole cell and none could be skipped. Compression is `zstd`. `source_granule` and `beam` are dictionary-encoded and hold one value per file, so provenance is close to free. Row-group boundaries and statistics here are illustrative; the row count is the real constant.

Column	Type	Meaning
native_lon, native_lat	double	position as published, never rewritten
native_height	double	elevation in the source's own datum
t	timestamp[ms]	observation time
source_granule, beam	dictionary	provenance, one value per file
source_chunk_index	int32	which fetch unit produced this row
<i>ATL06 adds</i>		
quality	int8	atl06_quality_summary
<i>ICESN adds</i>		
sn_slope, we_slope	double	along/across-track surface slope
<i>ICESAT2 (ATL03 photons) adds</i>		
row_id	uint64	$\text{crc32}(\text{granule}/\text{beam}) \ll 32$ photon index — a deterministic surrogate key
height_ref, native_frame	dictionary	e.g. WGS84 ellipsoid, ITRF2014
coreg_lon, coreg_lat, coreg_epoch	double	co-registered position at <code>COMMON_EPOCH = 2005.0</code> , materialised at ingest
signal_conf_landice	int8	photon confidence
photon_index	int64	index within the granule/beam

5 The metadata database

`data/index/meta.duckdb` holds two tables answering two *different* questions. Conflating them is the single most productive source of bugs in this subsystem.

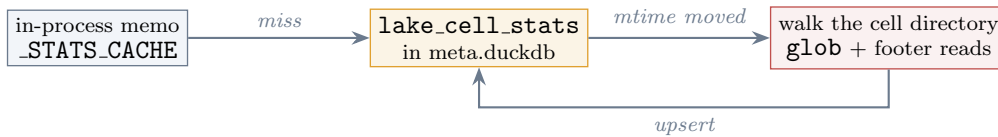
`coverage_cells` — *what has been fetched*. 118,157 rows.

<code>mission</code>	VARCHAR	ATL06 / GLAS / ICESN / ICESAT2
<code>granule, beam</code>	VARCHAR	<code>beam = na</code> where the source has no beams
<code>chunk_index</code>	INTEGER	
<code>h3_cell</code>	UBIGINT	
<code>ingested_at</code>	TIMESTAMP	
PK (mission, granule, beam, chunk_index, h3_cell)		

`lake_cell_stats` — *what is materialised*. One row per cell.

<code>mission, h3_cell</code>	VARCHAR, UBIGINT	PK
<code>dir_mtime_ns</code>	HUGEINT	the cell directory's mtime this rollup was computed at
<code>rows_known</code>	BOOLEAN	false when computed by a cheap pass that skipped row counts
<code>files, n_rows, n_bytes</code>	INTEGER, BIGINT, BIGINT	
<code>granules, beams</code>	VARCHAR[]	

Why `coverage_cells` cannot size the lake. It deliberately records cells a chunk *wanted* but wrote nothing to — `index_atl06` marks every wanted cell of a fetched chunk “so an all-fill cell is not re-fetched forever”, and the writer then drops non-finite positions and, for ATL06, segments with quality $\neq 0$. Measured on the deployed lake: **4,373 of 107,363 ATL06 rows (4.1%) name a file that does not exist**, over 184 cells and 922 granules — and every file on disk has a row (zero unmarked). Not drift, not eviction: a faithful record of a different fact. Hence the separate rollup table.



Gate at every tier is one `stat()` of the cell directory, so a steady-state poll opens no database at all. Cost on the box: 14.05 s $\rightarrow$ 0.27 s across a process restart, identical output.

6 Read paths and what they cost

Measured on the deployed `t3.large` in `us-west-2`, against 102,990 lake files and 71,640 index parquet.

Call	Walking files	From metadata	Source of the fast answer
<code>lake_summary(ATL06)</code>	7.09 s	0.27 s	<code>lake_cell_stats</code> + per-cell mtime
<code>index_status(ATL06)</code>	44.29 s	0.47 s	<code>_coverage/manifest.parquet</code>
<code>index_status(ATL03)</code>	0.05 s	0.01 s	
<code>index_status(GLAS)</code>	7.21 s	0.18 s	
<code>index_status(ICESN)</code>	22.95 s	0.11 s	
one page load	81.58 s	1.04 s	

The failure mode this table represents is worth naming: *work proportional to the store rather than to the request*. It has been found and fixed five times here. It is invisible on a developer machine, where the store is 100 $\times$ smaller, and it does not degrade gracefully — an endpoint slower than its own polling interval does not become slow, it becomes *never*, because the polls stack.

7 Should the rollup live in the H3 hierarchy?

The tempting design stores metadata at several resolutions — a `res-3` row summarising its `res-5` members — so a zoomed-out view reads few rows. Per §2 the *ID* hierarchy is exact even though the geometry is not, so this is well defined. The question is whether it pays.

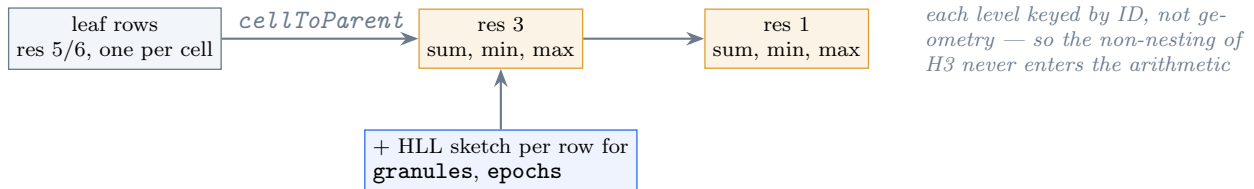
Quantity	Decomposable?	Why
<code>n_bytes</code> , <code>n_rows</code> , <code>files</code>	yes , sum	each file belongs to exactly one leaf cell
<code>ym_min</code> , <code>ym_max</code> , hence span	yes , min/max	min and max are associative
<code>granules</code> (distinct count)	no	one granule crosses many cells; summing children over-counts the shared ones
<code>epochs</code> (distinct <code>ym</code>)	no	same reason

This is not hypothetical: the client already rolls index cells up to the display grid with e.g. `+= x.g`, summing a distinct-granule count, so a granule crossing two res-5 cells is counted twice in their res-4 parent. The tooltip’s granule figure is an over-count at zoom levels coarser than the index resolution. Span and epochs, taken there as a `max`, are correct.

Recommendation: not yet, and not as stored rows.

1. **The leaf table is small enough to aggregate on demand.** `GROUP BY cellToParent(h3_cell, r)` over 185 lake cells or 14,482 index cells is microseconds in DuckDB. Materialising ancestors optimises something that is not slow.
2. **Invalidation multiplies.** Today’s correctness rests on one cheap, honest signal: the cell directory’s `mtime`. A stored ancestor has no directory and so no such signal; every leaf write would have to invalidate $O(\log)$ ancestors explicitly, and a writer that forgot would leave a wrong number with nothing to self-heal it. The current tiers degrade to a slow-but-correct walk; a stale ancestor degrades to a wrong answer.
3. **The distinct counts need sketches, not integers.** Doing it properly means an HLL sketch per cell per level so `granules` and `epochs` merge correctly — right at scale, disproportionate now.

The threshold to watch. Res-5 covers the globe in 2.0×10^6 cells; ATL06 occupies 14,482 of them at partial build. A whole-cryosphere index at res 5, or any move to res 7–8 for ATL03, puts the leaf count two to three orders of magnitude higher, and the on-demand `GROUP BY` stops being free. The migration is then:



8 Adding time to the index

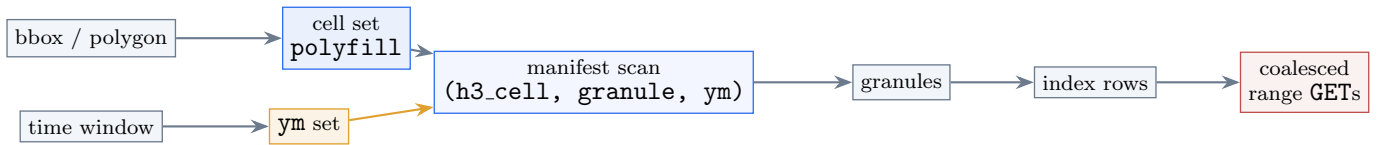
This section is speculative: a design sketch, not a description of what is built.

The lede. Sub-granule time is worth almost nothing, and the interesting work is all at the other end. An ATL06 granule is one RGT quarter-orbit — a few minutes of flight — and the part of it inside one res-5 cell is a few seconds. A time predicate finer than a granule therefore almost never eliminates a row that the *spatial* predicate has not already eliminated. What is missing is not resolution, it is *status*: time is a display field derived on the way out, not a predicate pushed into the scan, and it is derived four different ways for four missions.

What already exists. More than it looks like:

Where	Grain	Form
granule name (ATL0x)	second	ATL06_YYYYMMDDHHMMSS... — parsed, no I/O
gdate (GLAS, ICESSN)	day	an index column, on every row
cycle, rgt (ATL0x)	91-day repeat	index columns; the natural bucket for a repeat-track query
ym in manifest.parquet	month	<i>already</i> the third of the DISTINCT triple
delta_time	per segment	addressed by the chunk map, decoded only if fetched
t in the lake	millisecond	timestamp[ms], per point

So a spatio-temporal query is already answerable end to end from the manifest, at granule grain — but the planner does it by intersecting cells first and filtering granule *names* afterwards, in Python, after the whole spatial result has been materialised.



Today the gold branch does not exist: ym is read out of the scan for display, and the time filter is applied to granule names in Python afterwards. Making it an argument of the scan is the whole of step 1 below.

Step 1 — make ym a predicate, and partition the manifest by it. Push ym BETWEEN into the manifest scan, and write the rollup Hive-partitioned: `_coverage/ym=2019-03/part.parquet`, one COPY ... PARTITION_BY (ym) instead of one file. A time predicate then becomes directory pruning and the spatial one stays row-group pruning inside each part. This is the cheapest real win and it costs one line of the rebuild. The tradeoff to state plainly: a manifest sorted or partitioned by time gives up the cell locality a spatial-only scan enjoys. Partitioning rather than sorting is the compromise — coarse pruning on time, order preserved on cell within each partition.

Step 2 — one cross-mission space-time table. “Who observed this cell in 2019, from any mission?” means four manifest scans with four different date conventions today. A single `space_time(mission, h3_cell, ym, granules, n_rows)` table in `meta.duckdb`, produced by the same rollup pass, makes it one scan and normalises the date derivation in exactly one place. It is also the natural home for the elevation time-series work, which currently re-discovers coincident cells per query. Size is modest: the DISTINCT (mission, cell, ym) rollup drops the granule dimension that makes the ATL06 manifest 2.3 M rows.

Step 3 — a time bucket in the key, only if the leaf count explodes. Same argument as §7: do not materialise a hierarchy while the leaf GROUP BY is free. If it stops being free, the key becomes (h3_cell, bucket) — `cycle` for ICESat-2, `month` for the rest — and the real question is the sort order, since neither cell-major nor time-major dominates. The usual answer is a Z-order interleave of the two; the usual trap is assuming H3 IDs are spatially contiguous under integer order. They are not: an H3 ID packs a base-cell number and then a digit sequence, so integer order gives good locality *within* one of the 122 base cells and none across them. Interleaving `cellToParent(c,3)` with the time bucket is far better behaved than interleaving the raw 64-bit ID.

What I would not build. `t_min/t_max` on the index row. Two doubles per row is nothing in bytes, but populating them forces the builder to decode `delta_time` for every chunk — it currently decodes only `latitude` and `longitude` — and it buys a predicate the granule date already answers to within a few minutes.

What would overturn this. A collection whose granules span a long time: IceBridge campaign files, or any future gridded or seasonally-aggregated product. For those the granule date is a bad proxy and sub-granule time starts to earn its cost. If such a collection is added, revisit step 3 before step 1.

9 Invariants

Relied on throughout, and each violated at least once:

1. **Coverage is marked only after the file is on disk.** A crash loses cache, never truth.
2. **Index files land by rename.** Readers never see a torn Parquet, and the directory mtime is a sound change signal.
3. **All datasets of a beam share one chunk map.** `chunk_index` is a single integer naming the same segment span in every dataset. The builder checks chunk length and chunk count against `latitude` (ATL06) or `h_ph` (ATL03) and fails the build rather than assuming it.
4. **A lake file is written at the collection's own index resolution,** never at the planner's, so a partial cell is impossible by construction.
5. **Files are the source of truth for bytes and rows;** the metadata database is a cache of that truth, gated on a directory mtime, never the other way round.
6. **`coverage_cells` is a superset of the materialised set,** by design. Counting files from it over-counts by $\approx 4\%$.